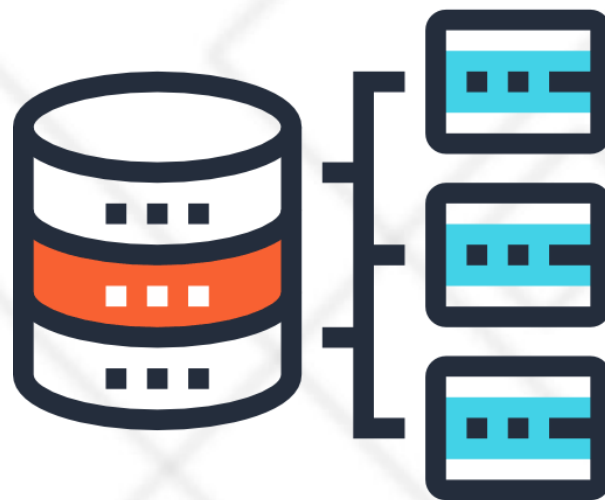




北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY



第9章 并发控制

北京理工大学 计算机学院

段春晖

duanch@bit.edu.cn



问题提出



□多用户数据库系统特征

- 允许多个用户同时使用的数据库系统
 - ◆ 飞机订票数据库系统
 - ◆ 银行数据库系统
- 特点：在同一时刻并发运行的事务数可达数百个

□多事务执行方式

(1) 事务串行执行

- ◆ 每个时刻只有一个事务运行，其他事务必须等到这个事务结束以后方能运行
- ◆ 不能充分利用系统资源和数据库资源

问题提出

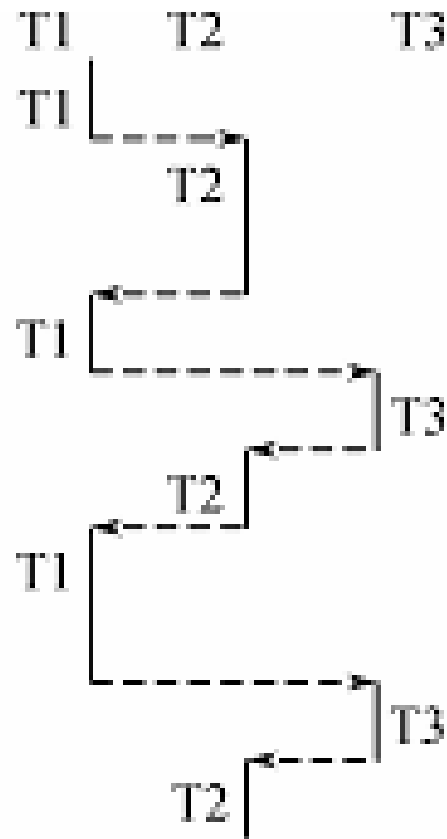
(2) 交叉并发方式

- ◆ 并事务的并行操作轮流交叉运行
- ◆ 是单处理机系统中的并发方式，能够减少处理机的空闲时间，提高系统的效率

(3) 同时并发方式

- ◆ 多处理机系统中，每个处理机可以运行一个事务，多个处理机可以同时运行多个事务，实现多个事务真正的并行运行
- ◆ 最理想的并发方式受制于硬件环境、更复杂的机制

□ 本章讨论的数据库系统并发控制技术是以单处理机系统为基础的





第9章 并发控制

- 9.1 并发事务运行存在的异常问题
- 9.2 并发调度的可串行性
- 9.3 基于封锁的并发控制技术
- 9.4 多粒度封锁
- 9.5 基于时间戳协议的并发控制
- 9.6 基于有效性确认的并发控制
- 9.7 插入与删除操作对并发控制的影响
- 9.8 SQL Server中的并发控制
- 9.9 小结

(回顾) 事务特征

□事务的四个特性:

- 原子性 (Atomicity)、一致性 (Consistency)、隔离性 (Isolation)、持续性 (Durability), 简称**ACID特性**

□1. 原子性 (Atomicity)

- 事务是数据库的逻辑工作单位, 事务中包括的诸操作要么都做, 要么都不做

□2. 一致性 (Consistency)

- 事务执行的结果必须是使数据库从一个一致性状态变到另一个一致性状态
- 一致性状态: 数据库中只包含成功事务提交的结果
- 不一致状态: 数据库中包含失败事务的结果

(回顾) 事务特征

□3. 隔离性

- 对并发执行而言，一个事务的执行不能被其他事务干扰

□4. 持续性

- 持续性也称永久性 (Permanence)
- 一个事务一旦提交，它对数据库中数据的改变就应该是永久性的。接下来的其他操作或故障不应该对其执行结果有任何影响

**事务并发执行可能会存取和存储不正确的数据，
破坏事务的隔离性和数据库的一致性**



9.1 并发事务运行存在的异常问题



□ 并发操作带来的数据不一致性

- 1. 丢失更新 (Lost Update)
- 2. 不可重复读 (Non-Repeatable Read)
- 3. 读“脏”数据 (Dirty Read)

1. 丢失更新

□以飞机订票系统为例

- 甲售票点事务 T_1 和乙售票点事务 T_2 同时读取某航班的机票余额 $R=100$ ；分别售出1张机票
- 结果明明卖出两张机票，数据库中机票余额只减少1

□丢失更新是指

- 事务 T_1 与事务 T_2 从数据库中读入同一数据并修改
- 事务 T_2 的提交结果破坏了事务 T_1 提交的结果，导致事务 T_1 的修改被丢失

T_1	T_2
① 读 $R=100$	读 $R=100$ $R=R-1$ 写回 $R=99$ 提交
②	
③ $R=R-1$ 写回 $R=99$ 提交	
④	

2. 不可重复读

□不可重复读是事务 T_1 读取数据后， T_2 对同一数据执行更新操作，使 T_1 再次读取该数据时，得到与前一次不同的值

T_1	T_2
① 读A=50 读B=100 求和=150	读B=100 $B \leftarrow B * 2$ 写回B=200
②	
③ 读A=50 读B=200 求和=250 (验算不对)	

三类不可重复读：

事务 T_1 读取某一数据后：

- T_2 对其做了修改，当 T_1 再次读该数据时，**得到与前一次不同的值**
- T_2 删除了其中部分记录，当 T_1 再次读取数据时，**某些记录消失**
- T_2 插入了一些记录，当 T_1 再次按相同条件读数据时，**多了一些记录**

后两种不可重复读有时也称为幻影现象 (Phantom Row)

3. 读“脏”数据

- 事务 T_1 修改某一数据，并将其写回磁盘
- 事务 T_2 读取同一数据后，事务 T_1 由于某种原因被撤销，这时事务 T_1 已修改过的数据恢复原值
- 事务 T_2 读到的数据就与数据库中的数据不一致，是不正确的数据，又称为“脏”数据

T_1	T_2
① 读$C=100$ $C \leftarrow C * 2$ 写回C ② ③ ROLLBACK C恢复为100	读$C=200$



9.1 并发事务运行存在的异常问题



□ 并发操作带来的数据不一致性

- 1. 丢失更新 (Lost Update)
- 2. 不可重复读 (Non-Repeatable Read)
- 3. 读“脏”数据 (Dirty Read)

□ 这种数据库的不一致性是由并发操作引起的，主要原因是**并发操作破坏了事务的隔离性**

- **并发控制机制要用正确的方式调度并发操作，使一个用户事务的执行不受其他事务的干扰，避免造成数据的不一致性**
 - ◆ 保证事务的隔离性
 - ◆ 保证数据库的一致性



第9章 并发控制

- 9.1 并发事务运行存在的异常问题
- 9.2 并发调度的可串行性
- 9.3 基于封锁的并发控制技术
- 9.4 多粒度封锁
- 9.5 基于时间戳协议的并发控制
- 9.6 基于有效性确认的并发控制
- 9.7 插入与删除操作对并发控制的影响
- 9.8 SQL Server中的并发控制
- 9.9 小结

9.2 并发调度的可串行性

- 计算机系统中并行事务中并行操作的调度是随机的，而不同的调度可能会产生不同的结果
- 将所有事务串行起来的调度策略是正确的调度策略
 - 如果一个事务运行过程中没有其他事务在同时运行，也就是说它没有受到其他事务的干扰，那么就可以认为该事务的运行结果是正常的或者预想的
 - 以不同的顺序串行执行事务也有可能产生不同的结果，但由于不会将数据库置于不一致状态，所以都可以认为是正确的
- 定义9.1 多个事务的并发执行是正确的，当且仅当并发执行的结果与这些事务按某一串行顺序执行的结果相同，这种调度策略被称为可串行化调度

可串行化是并发事务正确调度的准则

9.2.1 可串行化调度

□可串行性是并事务正确性的唯一准则

- 按这个准则规定，一个给定的并发调度，当且仅当它是可串行化的，才认为是正确调度

□例：现在有两个事务，分别包含下列操作：

事务1：读B； $A=B+1$ ； 写回A；

事务2：读A； $B=A+1$ ； 写回B；

假设A的初值为2，B的初值为2。

- 对这两个事务的不同调度策略

◆ 串行执行

- (a) 串行调度策略、(b) 串行调度策略

◆ 交错执行

- (c) 不可串行化的调度、(d) 可串行化的调度

9.2.1 可串行化调度

T_1	T_2
Slock B	
Y=B=2	
Unlock B	
Xlock A	
A=Y+1	
写回A(=3)	
Unlock A	
	Slock A
	X=A=3
	Unlock A
	Xlock B
	B=X+1
	写回B(=4)
	Unlock B

(a) 串行调度策略
正确的调度

9.2.1 可串行化调度

T ₁	T ₂	T ₁	T ₂
Slock B			SlockA
Y=B=2			X=A=2
Unlock B			Unlock A
Xlock A			Xlock B
A=Y+1			B=X+1
写回A(=3)			写回B(=3)
Unlock A			Unlock B
	Slock A	Slock B	
	X=A=3	Y=B=3	
	Unlock A	Unlock B	
	Xlock B	Xlock A	
	B=X+1	A=Y+1	
	写回B(=4)	写回A(=4)	
	Unlock B	Unlock A	

(a) 串行调度策略
正确的调度

(b) 串行调度策略
正确的调度

9.2.1 可串行化调度

T ₁	T ₂	T ₁	T ₂	T ₁	T ₂
Slock B Y=B=2 Unlock B Xlock A A=Y+1 写回A(=3) Unlock A			SlockA X=A=2 Unlock A Xlock B B=X+1 写回B(=3) Unlock B	Slock B Y=B=2 Unlock B Xlock A A=Y+1 写回A(=3) Unlock A	Slock A X=A=2 Unlock A Xlock B B=X+1 写回B(=3) Unlock B
	Slock A X=A=3 Unlock A Xlock B B=X+1 写回B(=4) Unlock B	Slock B Y=B=3 Unlock B Xlock A A=Y+1 写回A(=4) Unlock A			

(a) 串行调度策略
正确的调度

(b) 串行调度策略
正确的调度

(c) 不可串行化的调度
错误的调度

9.2.1 可串行化调度

□ 由于其执行结果与串行调度 (a) 的执行结果相同，所以是正确的调度

(d) 可串行化的调度
正确的调度

T_1	T_2
Slock B $Y=B=2$ Unlock B Xlock A $A=Y+1$ 写回A(=3) Unlock A	 Slock A 等待 等待 等待 $X=A=3$ Unlock A Xlock B $B=X+1$ 写回B(=4) Unlock B

9.2.1 可串行化调度

- DBMS必须提供一定手段来保证调度是可串行化的
- 如何保证并发操作的调度是正确的?
 - 从理论上讲，在某一事务执行时禁止其他事务执行的调度策略一定是可串行化的调度
 - ◆ 这也是最简单的调度策略，但这种方法实际上是不可行的，因为它不利于共享

9.2.2 调度的冲突等价性

□冲突操作

- 冲突操作是指不同的事务对同一个数据的读写操作和写写操作

◆ $R_i(x)$ 与 $W_j(x)$ /* 事务 T_i 读 x , T_j 写 x */

◆ $W_i(x)$ 与 $W_j(x)$ /* 事务 T_i 写 x , T_j 写 x */

- 其他操作是不冲突操作
- 不同事务的冲突操作和同一事务的两个操作不能交换 (Swap)

□定义9.2 如果一个调度 S 能通过一系列非冲突操作执行顺序的交换变成调度 S_1 , 则称调度 S 和 S_1 冲突等价

9.2.2 调度的冲突等价性

□[例 9-3] 证明调度S是否是可串行化调度

- $S = R_1(A)W_1(A)R_2(A)W_2(A)R_1(B)W_1(B)R_2(B)W_2(B)$
- 把 $W_2(A)$ 与 $R_1(B)W_1(B)$ 交换, 得到:

$R_1(A)W_1(A)R_2(A)R_1(B)W_1(B)W_2(A)R_2(B)W_2(B)$

T_1	T_2
Read(A)	
Write(A)	
	Read(A)
	Write(A)
Read(B)	
Write(B)	
	Read(B)
	Write(B)

9.2.2 调度的冲突等价性

□[例 9-3] 证明调度 S 是否是可串行化调度

- $S=R_1(A)W_1(A)R_2(A)W_2(A)R_1(B)W_1(B)R_2(B)W_2(B)$
- 把 $W_2(A)$ 与 $R_1(B)W_1(B)$ 交换, 得到:

$$R_1(A)W_1(A)R_2(A)R_1(B)W_1(B)W_2(A)R_2(B)W_2(B)$$

- 再把 $R_2(A)$ 与 $R_1(B)W_1(B)$ 交换:

$$L=R_1(A)W_1(A)R_1(B)W_1(B)R_2(A)W_2(A)R_2(B)W_2(B)$$

- 因为 L 等价于一个串行调度 T_1, T_2
- 所以调度 S 是可串行化的调度

L 和 S 是 (冲突) 等价的

9.2.3 调度的状态等价性

- 定义9.4 我们称一个调度是状态可串行的，如果它的状态等价于一个串行调度

- 可串行化调度的充分条件
 - 一个调度 S 在保证冲突操作的次序不变的情况下，通过交换两个事务不冲突操作的次序得到另一个调度 S' ，如果 S' 是串行的，称调度 S 为冲突可串行化的调度

9.2.3 调度的状态等价性

□ 一个调度是冲突可串行化，一定是状态可串行的
(可串行化的调度)

- 冲突可串行化调度是可串行化调度的充分条件，不是必要条件。还有不满足冲突可串行化条件的可串行化调度

□ [例] 有3个事务

$$T_1 = W_1(Y)W_1(X), \quad T_2 = W_2(Y)W_2(X), \quad T_3 = W_3(X)$$

- 调度 $L_1 = W_1(Y)W_1(X)W_2(Y)W_2(X)W_3(X)$ 是一个串行调度
- 调度 $L_2 = W_1(Y)W_2(Y)W_2(X)W_1(X)W_3(X)$ 不满足冲突可串行化。因为每对操作都是冲突的，不能交换
- 但是调度 L_2 是可串行化的。因为 L_2 执行的结果与调度 L_1 相同，Y 的值都等于 T_2 的值，X 的值都等于 T_3 的值

第9章 并发控制

- 9.1 并发事务运行存在的异常问题
- 9.2 并发调度的可串行性
- 9.3 基于封锁的并发控制技术
- 9.4 多粒度封锁
- 9.5 基于时间戳协议的并发控制
- 9.6 基于有效性确认的并发控制
- 9.7 插入与删除操作对并发控制的影响
- 9.8 SQL Server中的并发控制
- 9.9 小结

9.3 基于封锁的并发控制技术



□ 并发控制的主要技术有

- 封锁 (Locking)
- 时间戳 (Timestamp)
- 乐观控制法

□ 并发事务的可串行化技术，实质上遵循了一种约束：

- 即当一个事务存取某一个数据项时，不允许其它事务修改这一数据项。这就是封锁技术 (Locking)

□ 商用的DBMS一般都采用封锁方法

9.3 基于封锁的并发控制技术



□封锁是实现并发控制的一个非常重要的技术

- 事务T在对某个数据对象（如表、记录等）操作之前
 - ◆ 先向系统发出请求，对其加锁
- 加锁后
 - ◆ 事务T就对该数据对象有了一定的控制
- 在事务T释放它的锁之前
 - ◆ 其它的事务不能更新此数据对象

□一个事务对某个数据对象加锁后究竟拥有什么样的控制由封锁的类型决定

9.3.1 锁

□确切的控制是由封锁的类型决定的

- 共享锁 (Share lock, 简记为S锁, 又称为读锁)

- ◆ 若事务T对数据对象A加上S锁, 则其它事务只能再对A加S锁, 而不能加X锁, 直到T释放A上的S锁
- ◆ 保证其他事务可以获得对象A的S锁、读取A, 但在T释放A上的S锁之前不能对A做任何修改

- 排它锁 (eXclusive lock, 简记为X锁, 又称为写锁)

- ◆ 若事务T对数据对象A加上X锁, 则只允许T读取和修改A, 其它任何事务都不能再对A加任何类型的锁, 直到T释放A上的锁

9.3.1 锁

□锁的相容矩阵

Y, 相容的请求
N, 不相容的请求

$T_1 \backslash T_2$	X	S	-
X	N	N	Y
S	N	Y	Y
-	Y	Y	Y

- 列表示事务 T_1 已经获得的数据对象上的锁的类型，其中横线表示没有加锁
- 行表示事务 T_2 对同一数据对象发出的封锁请求

□ T_2 的封锁请求能否被满足用矩阵中的Y和N表示

- Y表示事务 T_2 的封锁要求与 T_1 已持有的锁相容，封锁请求可以满足
- N表示 T_2 的封锁请求与 T_1 已持有的锁冲突， T_2 的请求被拒绝

9.3.2 封锁协议

□封锁协议

- 运用封锁方法时，对数据对象加锁时需要约定一些规则
 - ◆ 何时申请封锁
 - ◆ 持锁时间
 - ◆ 何时释放封锁等

9.3.2 封锁协议

1. 一级封锁协议

- 事务 T 在修改数据 A 前必须先对其加 X 锁，直到事务结束才释放

- 符合一级封锁协议
- 没有丢失修改

T_1	T_2
① Xlock A 获得	
② 读 $A=16$	
③ $A \leftarrow A-1$ 写回 $A=15$ Commit Unlock A	Xlock A 等待 等待 等待 等待
④	获得 Xlock A 读 $A=15$ $A \leftarrow A-1$ 写回 $A=14$ Commit Unlock A
⑤	

1. 一级封锁协议

□一级封锁协议在读数据时不加锁，所以不能保证可重复读和不读“脏”数据

T ₁	T ₂
① Xlock A 获得	读A=15
② 读A=16 A←A-1 写回A=15	
③	
④ Rollback Unlock A	

读“脏”数据

不可重复读

T ₁	T ₂
① 读A=50 读B=100 求和=150	Xlock B 获得 读B=100 B←B*2 写回B=200 Commit Unlock B
②	
③ 读A=50 读B=200 求和=250 (验算不对)	

2. 二级封锁协议

□二级封锁协议规定：

- 在一级封锁协议基础上，事务 T 在读数据 A 之前必须先对其加 S 锁，读完后即可释放 S 锁
- 增加二级封锁协议的目的是防止读“脏”数据

T_1	T_2
① Xlock C 读 $C = 100$ $C \leftarrow C * 2$ 写回 $C = 200$	
②	
③ ROLLBACK (C 恢复为100) Unlock C	Slock C 等待 等待 等待 等待 获得 Slock C 读 $C = 100$
④	
⑤	Unlock C

2. 二级封锁协议

□ 二级封锁协议并不能保证避免不可重复读的问题

- 因为事务 T_1 在读数据 B 之前加上的S锁，读完后即释放了，以后再读时有可能数据发生了变化
- 解决的方法是，加在数据 B 上的S锁直到事务结束才释放

T_1	T_2
① Slock A 读A=50 Unlock A Slock B 读B=100 Unlock B 求和=150	
②	Xlock B 获得 读B=100 $B \leftarrow B * 2$ 写回B=200 Commit Unlock B
③ Slock A 读A=50 Unlock A Slock B 读B=200 求和=250 (验算不对)	

- 在二级封锁协议基础上，某一事务施加的S锁要保持到该事务结束时才释放

35

9.3.3 活锁

□封锁技术可以有效地解决并行操作的一致性问题，但也带来一些新的问题：死锁，活锁

- **活锁**：在数据库系统中活锁是指某个事务由于请求封锁，但总也得不到锁而长时间处于等待状态

T ₁	T ₂	T ₃	T ₄
lock R	.	.	.
.	lock R	.	.
.	等待	Lock R	.
Unlock	等待	.	Lock R
.	等待	Lock R	等待
.	等待	.	等待
.	等待	Unlock	等待
.	等待	.	Lock R
.	等待	.	.

9.3.3 活锁

□ 如何避免活锁

- 采用先来先服务的策略：
 - ◆ 当多个事务请求封锁同一数据对象时，按请求封锁的先后次序对这些事务排队
 - ◆ 该数据对象上的锁一旦释放，首先批准申请队列中第一个事务获得锁

9.3.4 死锁

□死锁是指在同时处于等待状态的两上或多个事务中相互封锁了对方请求的资源，使得没有任何一个事务可以获得足够的资源运行完毕，而永远等待下去

T ₁	T ₂
Xlock R ₁	.
.	.
.	Xlock R ₂
.	.
Xlock R ₂	.
等待	Xlock R ₁
等待	等待
等待	等待
.	.

□解决死锁的方法

- 1. 预防死锁
- 2. 允许死锁发生
 - ◆ 死锁的诊断与解除

1. 预防死锁

□分析产生死锁的原因

- 两个或多个事务都已封锁了一些数据对象
- 然后又都请求对已被其他事务封锁的数据对象加锁
- 出现死等待

□预防死锁的发生就是要破坏产生死锁的条件

□预防死锁的方法

- 一次封锁法
- 顺序封锁法
- 事务重试法

(1) 一次封锁法

□ 解决方法

- 将事务在执行过程中可能要封锁的数据对象全部加锁

□ 存在的问题

- 将以后要用到的全部数据加锁，势必扩大了封锁的范围，从而降低了系统的并发度
- 数据库中数据是不断变化的，原来不要求封锁的数据，在执行过程中可能会变成封锁对象，所以很难事先精确地确定每个事务所要封锁的数据对象

(2) 顺序封锁法

- 顺序封锁法是预先对数据对象规定一个封锁顺序，所有事务都按这个顺序实行封锁
- 顺序封锁法存在的问题
 - 数据库系统中可封锁的数据对象极其众多，并且随数据的插入、删除等操作而不断地变化，要维护这样极多而且变化的资源的封锁顺序非常困难，成本很高。
维护成本高
 - 事务的封锁请求可以随着事务的执行而动态地决定，很难事先确定每一个事务要封锁哪些对象，因此也就很难按规定的顺序去施加封锁。难于实现
 - ◆ 例：规定数据对象的封锁顺序为A, B, C, D, E。事务 T_3 起初要求封锁数据对象B, C, E，但当它封锁了B, C后，才发现还需要封锁A，这样就破坏了封锁顺序

(3) 事务重试法

□使用抢占机制和事务回滚

□例如，当事务 T_2 申请的锁已被事务 T_1 占有时

- 根据事务开始的先后
- 将 T_1 释放的锁授予 T_2
- 而事务 T_1 回滚后自动重试

□结论

- 在操作系统中广为采用的预防死锁的策略并不很适合数据库的特点
- DBMS在解决死锁的问题上更普遍采用的是诊断并解除死锁的方法

2. 死锁的检测与恢复

□ 允许死锁发生

□ 解除死锁

- 由DBMS的并发控制子系统定期检测系统中是否存在死锁
- 一旦检测到死锁，就要设法解除
 - ◆ 选择一个处理死锁代价最小的事务，将其撤销，释放此事务持有的所有的锁，使其它事务能继续运行下去
 - ◆ 被撤销的事务需要回滚

2. 死锁的检测与恢复

□(1) 超时法

- 如果一个事务的等待时间超过了规定的时限，就认为发生了死锁
- 优点
 - ◆ 实现简单
- 缺点
 - ◆ 有可能误判死锁
 - ◆ 时限若设置得太长，死锁发生后不能及时发现

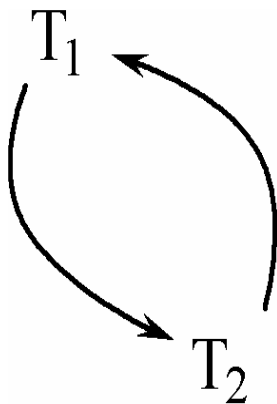
2. 死锁的检测与恢复

□(2) 事务等待图法

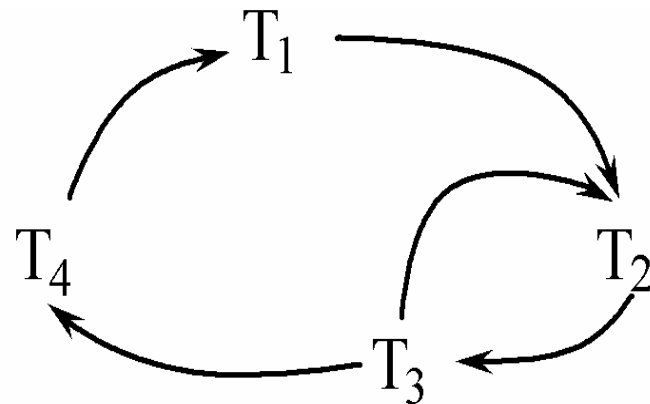
- 用事务等待图动态反映所有事务的等待情况
- 事务等待图是一个有向图 $G=(V, U)$
 - ◆ V 为结点的集合, 每个结点表示正运行的事务
 - ◆ U 为边的集合, 每条边表示事务等待的情况
- 若 V_1 等待 V_2 , 则 V_1 、 V_2 之间画一条有向边, 从 V_1 指向 V_2
- 并发控制子系统周期性地 (比如每隔1min) 检测事务等待图, 如果发现图中存在回路, 则表示系统中出现了死锁

2. 死锁的检测与恢复

- 图(a)中, 事务 T_1 等待 T_2 , T_2 等待 T_1 , 产生了死锁
- 图(b)中, 事务 T_1 等待 T_2 , T_2 等待 T_3 , T_3 等待 T_4 , T_4 又等待 T_1 , 产生了死锁
- 图(b)中, 事务 T_3 可能还等待 T_2 , 在大回路中又有小回路



(a)



(b)

事务等待图

2. 死锁的检测与恢复

□死锁的恢复

- 解除死锁的方法是回滚一个或多个相关事务
 - ◆ 选择一个处理死锁代价最小的事务，将其撤销
 - ◆ 释放此事务持有的所有的锁，使其它事务能继续运行下去

9.3.5 两阶段封锁协议

- **两阶段封锁协议** (Two-Phase Locking, 简称2PL) 是最常用的一种封锁协议
 - 理论上可以证明使用两段封锁协议产生的是可串行化调度
- **两阶段封锁协议**指所有事务必须分两个阶段对数据项加锁和解锁
 1. 在对任何数据进行读、写操作之前, 事务首先要获得对该数据的封锁
 2. 在释放一个封锁之后, 事务不再获得任何其他封锁
- **两阶段封锁协议**是保证并发调度可串行性的封锁协议

9.3.5 两阶段封锁协议

□“两阶段”锁的含义：事务分为两个阶段

- 第一阶段是获得封锁，也称为扩展阶段

- ◆ 在这阶段，事务可以申请获得任何数据项上的任何类型的锁，但是不能释放任何锁

- 第二阶段是释放封锁，也称为收缩阶段

- ◆ 在这阶段，事务可以释放任何数据项上的任何类型的锁，但是不能再申请任何锁

□例：事务1遵守两阶段封锁协议，事务2不遵守两阶段封锁协议

Slock A ... Slock B ... Xlock C ... Unlock B ... Unlock A ... Unlock C

Slock A ... Unlock A ... Slock B ... Xlock C ... Unlock C ... Unlock B

9.3.5 两阶段封锁协议

- 并行执行的所有事务均遵守两阶段封锁协议，则对这些事务的所有并行调度策略都是可串行化的



所有遵守两阶段封锁协议的事务，其并行执行的结果一定是正确的

- 事务遵守两段锁协议是可串行化调度的充分条件，而不是必要条件
 - 可串行化的调度中，不一定所有事务都必须符合两阶段封锁协议

T₁	T₂
Slock B	
读B=2	
Y=B	
Xlock A	
	Slock A
A=Y+1	等待
写回A=3	等待
Unlock B	等待
Unlock A	等待
	Slock A
	读A=3
	Y=A
	Xlock B
	B=Y+1
	写回B=4
	Unlock B
	Unlock A

(a) 遵守两段锁协议

T ₁	T ₂	T ₁	T ₂
Slock B		Slock B	
读B=2		读B=2	
Y=B		Y=B	
Xlock A		Unlock B	
	Slock A	Xlock A	Slock A
A=Y+1	等待		等待
写回A=3	等待		等待
Unlock B	等待	A=Y+1	等待
Unlock A	等待	写回A=3	等待
	Slock A	Unlock A	Slock A
	读A=3		读A=3
	Y=A		X=A
	Xlock B		Unlock A
	B=Y+1		Xlock B
	写回B=4		B=X+1
	Unlock B		写回B=4
	Unlock A		Unlock B

(a) 遵守两段锁协议

(b) 不遵守两段锁协议

9.3.5 两阶段封锁协议

□ 两阶段封锁协议与防止死锁的一次封锁法

- 一次封锁法要求每个事务必须一次将所有要使用的数据全部加锁，否则就不能继续执行，因此**一次封锁法遵守两段锁协议**
- 但是两阶段封锁协议并不要求事务必须一次将所有要使用的数据全部加锁，因此**遵守两阶段封锁协议的事务可能发生死锁**

**遵守两阶段封锁协议
的事务发生死锁**

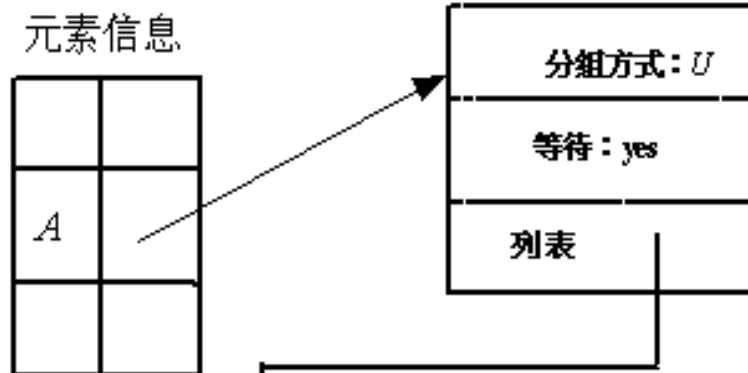
T ₁	T ₂
Slock B 读B=2	Slock A 读A=2
Xlock A 等待 等待	Xlock B 等待

9.3.6 锁表

- 锁的请求、授予和解除是由数据库系统的锁管理器 (Lock Manager) 完成
 - 针对锁请求消息返回授予锁消息或者发生死锁时要求事务回滚的消息
 - 解除锁消息只需要一个确认应答
- 锁管理器为目前已加锁的数据项维护一个记录链表，每个锁请求为一个记录，按请求的到达顺序排序这个表称为锁表 (Lock Table)

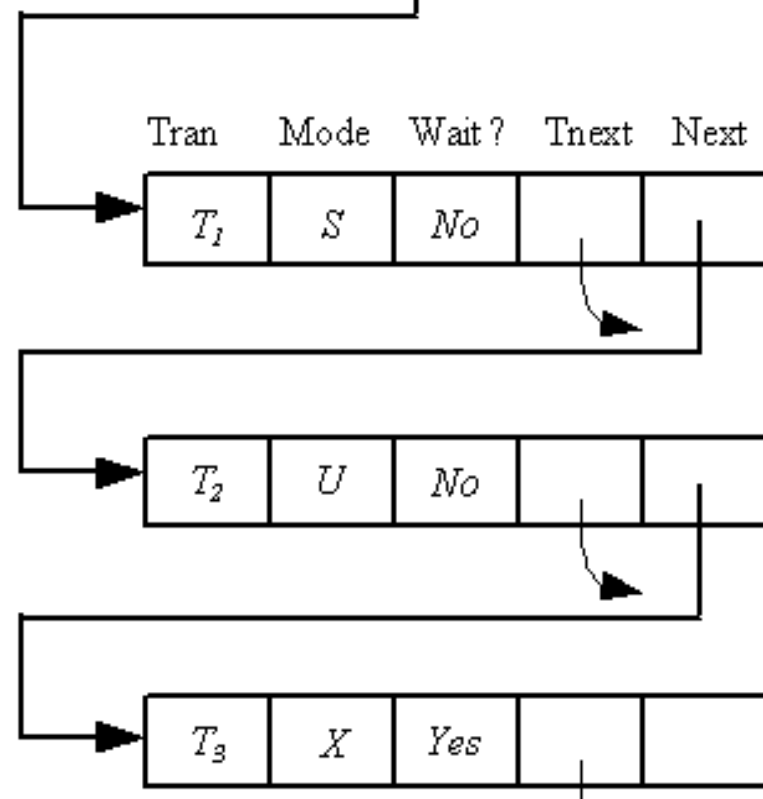
9.3.6 锁表

□锁表将数据库元素和封锁信息联系在一起



□封锁请求处理

- 假设事务 *T* 请求 *A* 上的锁
- 如果没有 *A* 的锁表项, 则表明 *A* 无锁, 因此相应的表项被创建
- 如果存在 *A* 的锁表项, 则依据它决定封锁请求



□解锁处理

第9章 并发控制

- 9.1 并发事务运行存在的异常问题
- 9.2 并发调度的可串行性
- 9.3 基于封锁的并发控制技术
- 9.4 多粒度封锁
- 9.5 基于时间戳协议的并发控制
- 9.6 基于有效性确认的并发控制
- 9.7 插入与删除操作对并发控制的影响
- 9.8 SQL Server中的并发控制
- 9.9 小结

9.4 多粒度封锁

□ 1. 什么是封锁粒度

- 封锁对象的大小称为封锁粒度
- 封锁对象可以是逻辑单元（属性至数据库），也可以是物理单元（数据页）

例：在关系数据库中，封锁对象：

- ◆ 逻辑单元：属性值、属性值集合、元组、关系、索引项、整个索引、整个数据库等
- ◆ 物理单元：页（数据页或索引页）、物理记录等

□ **多粒度封锁** (Multiple Granularity Locking)：在一个系统中同时支持多种封锁粒度供不同的事务选择

9.4 多粒度封锁

□2. 选择封锁粒度的原则

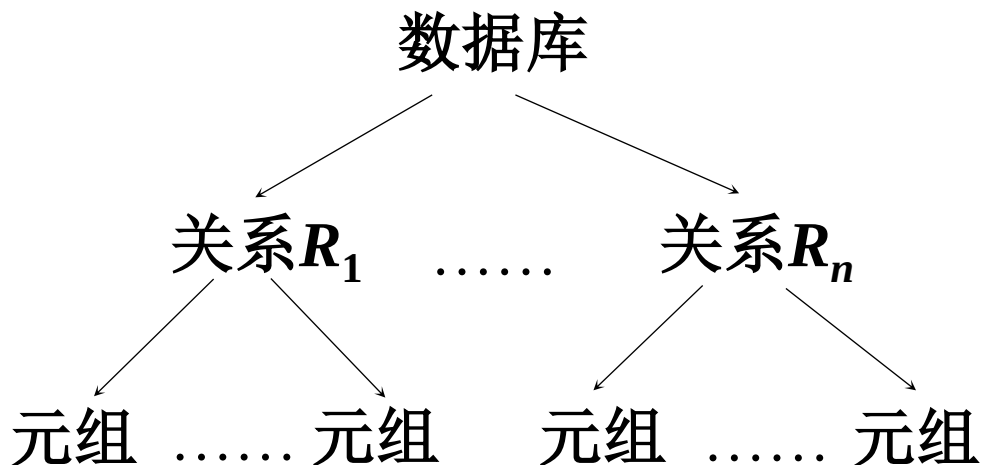
- 封锁的粒度越 大, 小
- 系统被封锁的对象 少, 多
- 并发度 小, 高
- 系统开销 小, 大
- 选择封锁粒度考虑封锁机构和并发度两个因素, 对系统开销与并发度进行权衡
 - ◆ 需要处理多个关系的大量元组的用户事务: 以数据库为封锁单位
 - ◆ 需要处理大量元组的用户事务: 以关系为封锁单元
 - ◆ 只处理少量元组的用户事务: 以元组为封锁单位

9.4 多粒度封锁

□多粒度树

- 以树形结构来表示多级封锁粒度
- 根结点是整个数据库，表示最大的数据粒度
- 叶结点表示最小的数据粒度

□例：三级粒度树。根结点为数据库，数据库的子结点为关系，关系的子结点为元组



1. 多粒度封锁

□多粒度封锁协议

- 允许多粒度树中的每个结点被独立地加锁
- 对一个结点加锁意味着这个结点的所有后裔结点也被加以同样类型的锁
- 在多粒度封锁中一个数据对象可能以两种方式封锁，封锁的效果是一样的
 - ◆ 显式封锁：直接加到数据对象上的封锁
 - ◆ 隐式封锁：由于其上级结点加锁而使该数据对象加上了锁

1. 多粒度封锁

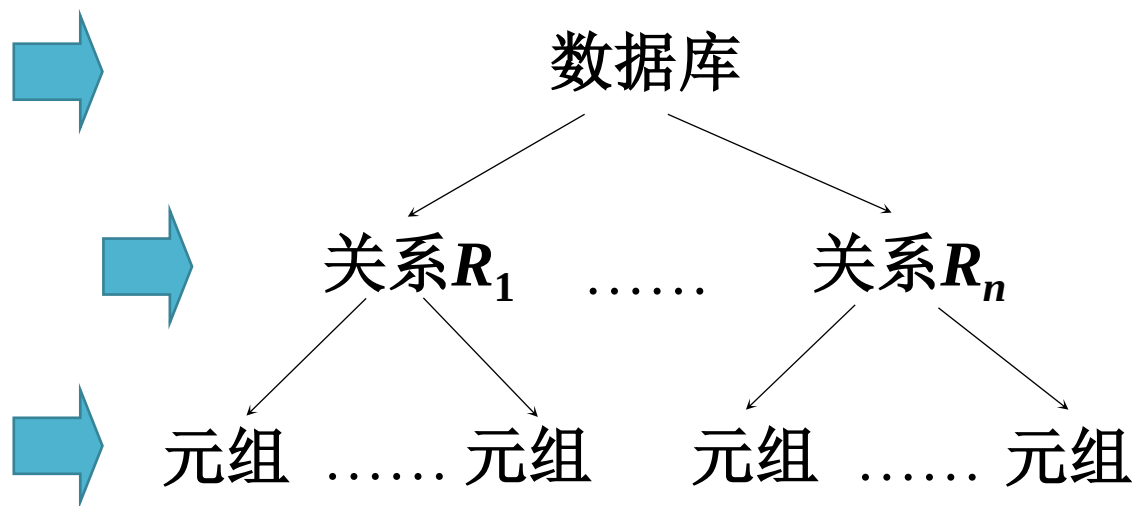
□ 系统检查封锁冲突时 怎么办?

- 要检查显式封锁、还要检查隐式封锁
- 对某个数据对象加锁时系统检查的内容
 - ◆ 该数据对象：
 - 有无显式封锁与之冲突
 - ◆ 所有上级结点
 - 检查本事务的显式封锁是否与该数据对象上的隐式封锁冲突 —— 由上级结点封锁造成的
 - ◆ 所有下级结点
 - 检查本事务的隐式封锁是否与该数据对象上的显式封锁冲突 —— 将加到下级结点的封锁

1. 多粒度封锁

□ 例如事务 T 要对关系 R_1 加X锁

- 系统必须搜索其上级结点数据库、关系 R_1
- 还要搜索 R_1 的下级结点, 即 R_1 中的每一个元组
- 如果其中某一个数据对象已经加了不相容锁, 则 T 必须等待



2. 意向锁

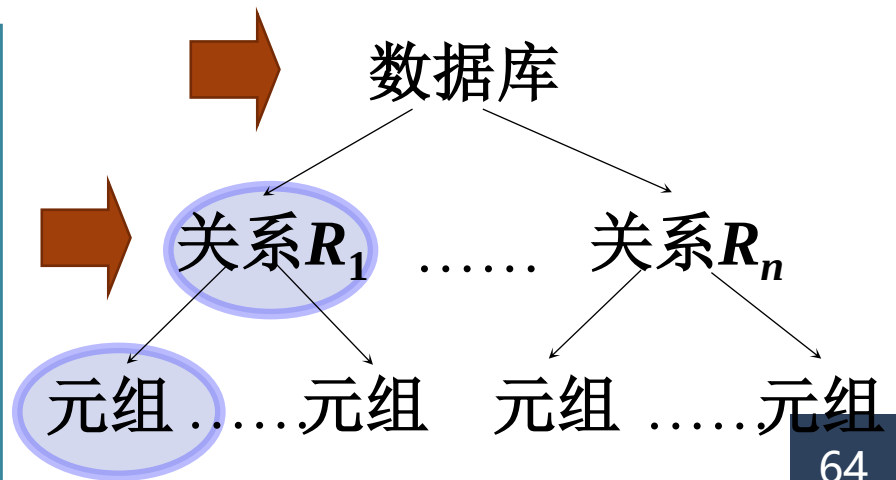
□ 引进意向锁 (Intention Lock) 目的

- 提高对某个数据对象加锁时系统的检查效率

□ 什么是意向锁

- 对任一结点加基本锁，必须先对它的上层结点加意向锁。如果对一一个结点加意向锁，则说明该结点的下层结点正在被加锁
- 例：对任一元组 t 加锁，先对数据库和关系 R 加意向锁

- 事务 T 要对关系 R 加 S 锁，系统只要检查根结点数据库和关系 R 是否已加了不相容的锁
- 不需要搜索和检查 R 中的每一个元组是否加了 X 锁





常用意向锁



□意向共享锁 (Intent Share Lock, 简称IS锁)

- 如果对一个数据对象加IS锁, 表示它的后裔结点拟(意向)加S锁。例: 要对某个元组加S锁, 则要先对关系和数据库加IS锁

□意向排它锁 (Intent Exclusive Lock, 简称IX锁)

- 如果对一个数据对象加IX锁, 表示它的后裔结点拟(意向)加X锁。例: 要对某个元组加X锁, 先要对关系和数据库加IX锁

□共享意向排它锁 (Share Intent Exclusive Lock, 简称SIX锁)

- 如果对一个数据对象加SIX锁, 表示对它加S锁, 再加IX锁, 即 $SIX = S + IX$ 。例: 对某个表加SIX锁, 则表示该事务要读整个表(所以要对该表加S锁), 同时会更新个别元组(所以要对该表加IX锁)

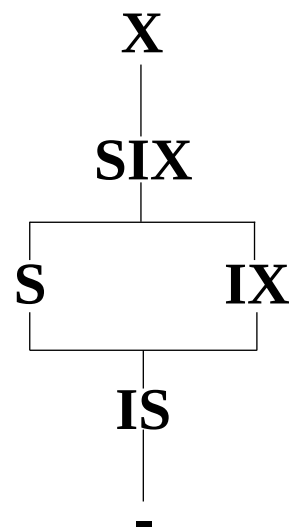
2. 意向锁

意向锁的相容矩阵

T ₁ \ T ₂	S	X	IS	IX	SIX	-
S	Y	N	Y	N	N	Y
X	N	N	N	N	N	Y
IS	Y	N	Y	Y	Y	Y
IX	N	N	Y	Y	N	Y
SIX	N	N	Y	N	N	Y
-	Y	Y	Y	Y	Y	Y

Y=Yes, 表示相容的请求

N=No, 表示不相容的请求



锁的强度的偏序关系

锁的强度

- 锁的强度是指它对其他锁的排斥程度
- 一个事务在申请封锁时以强锁代替弱锁是安全的，反之则不然

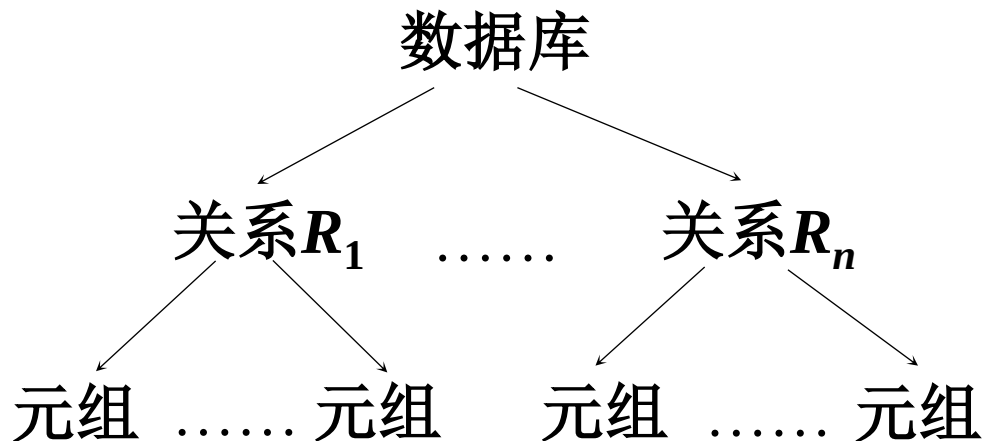
2. 意向锁

□ 具有意向锁的多粒度封锁方法

- 申请封锁时应该按自上而下的次序进行
- 释放封锁时应该按自下而上的次序进行

□ 例如：事务 T_1 要对关系 R_1 加S锁

- 要首先对数据库加IS锁
- 检查数据库和 R_1 是否已加了不相容的锁 (X或IX)
- 不再需要搜索和检查 R_1 中的元组是否加了不相容的锁 (X锁)



2. 意向锁

□ 具有意向锁的多粒度封锁方法

- 提高了系统的并发度
- 减少了加锁和解锁的开销
- 在实际的数据库管理系统产品中得到广泛应用

第9章 并发控制

- 9.1 并发事务运行存在的异常问题
- 9.2 并发调度的可串行性
- 9.3 基于封锁的并发控制技术
- 9.4 多粒度封锁
- 9.5 基于时间戳协议的并发控制
- 9.6 基于有效性确认的并发控制
- 9.7 插入与删除操作对并发控制的影响
- 9.8 SQL Server中的并发控制
- 9.9 小结

9.8 SQL Server中的并发控制



- SQL Server支持事务处理以管理多个事务
 - 事务在遇到COMMIT TRANSACTION之前，不会对数据库造成永久性的改变
 - 事务在遇到ROLLBACK TRANSACTION语句时，可以撤销它所作的改变
- SQL Server在事务执行期间动态设置和调整适当的锁定级别，也可以手动控制如何使用锁
- SQL Server中有两种事务
 - 隐性事务
 - 显式事务（或称为用户定义事务）
- 提交的事务不能撤销或回滚

事务恢复和检查点

□事务恢复

- SQL Server可以在发生故障时利用日志自动恢复数据
 - ◆ 使用事务日志重作所有已提交的事务
 - ◆ 回滚任何未提交的事务

□初始时，在数据缓存中和磁盘上的页是相同的

- 当事务提交时，改变数据缓存内的页
- 当缓存满时，改变过的页写入磁盘
- 当到达检查点时，缓存写入磁盘。磁盘数据再次与缓存数据保持一致

□事务提交后，对数据库的修改是否立即记录到磁盘上？

- 不是。事务提交后，只改变数据缓存的内容，到达检查点时才写入磁盘

SQL Server的锁定



□ SQL Server可以锁定以下项目类型

项目	描述
RID	行标识符。用于单独锁定表中的一行
键	索引中的行锁。用于保护可串行事务中的键范围
页	8KB的数据页或索引页
扩展盘区	相邻的八个数据页或索引页构成的一组，在空间分配中使用
表	包括所有数据和索引在内的整个表
数据库	整个数据库，在数据库的还原中使用

- SQL Server锁的类型：基本锁和特殊情况锁
- 基本锁：包括共享锁和排它锁
 - 读操作获得共享锁，写操作获得排它锁
 - ◆ 共享锁：用于不更改或不更新数据的操作（只读操作），如SELECT语句
 - ◆ 排它锁：用于数据修改操作，例如INSERT、UPDATE或删除DELETE
- 特殊情况锁
 - 意向锁、更新锁、架构锁、大容量更新锁
- 可使用sp_lock命令查看锁的信息

- 意向锁：SQL Server内部使用意向锁，以使锁定冲突减至最少
 - 意向锁是为了减少锁定冲突而设的，它建立了锁的层次结构，使其它事务不能在所有包括锁的更高层次上获得锁
 - 意向锁包括意向共享 (IS)、意向排它 (IX) 和意向排它共享 (SIX)。它并不是单独的锁模式，而是对其它锁起限定作用
 - 意向锁的设定目的：由于SQL Server可以在不同粒度级别上获得锁，需要一个标识资源的一部分已经被锁定的机制



锁的类型



- **更新锁**：用于可更新的资源中，防止当多个会话在读取、锁定及随后可能进行的资源更新时发生常见形式的死锁
 - 更新锁是共享锁和排它锁的混合，能提高并发度
 - 一个资源上面只能有一个更新锁，其他共享锁可以和更新锁同时存在
 - 但如果事务要修改数据，则将共享锁升级为排它锁，这时候其他的共享锁不能使用
 - 在未升级成排它锁之前，更新锁和共享锁是兼容的



锁的类型



- 架构锁：在执行依赖于表架构的操作时使用，确保表或索引在被另外的会话引用时不被删除或更改架构。架构锁包括架构稳定性 (Sch-S) 锁和架构修改 (Sch-M) 锁
- 大容量更新锁：向表中大容量复制数据并指定了 TABLOCK 提示时使用
 - 大容量更新锁允许进程将数据并发地大容量复制到同一表，同时防止其他不进行大容量复制数据的进程访问该表。当将数据大容量复制到表，且指定了 TABLOCK 提示或者使用 sp_tableoption 设置了 table lock on bulk 表选项时，将使用大容量更新锁

事务的隔离级别

- 尽管可串行性对于事务确保数据库中的数据在所有时间内的正确性相当重要，然而许多事务并不总是要求完全的隔离
- 事务准备接受不一致数据的级别称为隔离级别
- 隔离级别是一个事务必须与其它事务进行隔离的程度
 - 较低的隔离级别可以增加并发，但代价是降低数据的正确性
 - 较高的隔离级别可以确保数据的正确性，但可能对并发产生负面影响
- 应用程序要求的隔离级别确定了SQL Server使用的锁定行为

事务的隔离级别

- SQL-92定义了下列四种隔离级别，SQL Server支持所有这些隔离级别：
 - 未提交读（事务隔离的最低级别，仅可保证不读取物理损坏的数据）
 - 提交读（SQL Server默认级别）
 - 可重复读
 - 可串行读（事务隔离的最高级别，事务之间完全隔离）
- 如果事务在可串行读隔离级别上运行，则可以保证任何并发重叠事务均是串行的

事务的隔离级别



选项	描述
READ COMMITTED	提交读。在读取时使用共享锁，不允许脏读
READ UNCOMMITTED	未提交读。不使用共享锁，忽略排他锁，允许脏读
REPEATABLE READ	可重复读。不可能发生脏读和不可重复读取，保持读锁直到事务结束
SERIALIZABLE	防止其他用户更新或插入符合本事务中WHERE子句条件的行。不可能发生幻像读

事务的隔离级别列表

□ 下面四种隔离级别允许不同类型的行为

隔离级别	脏读	不可重复读	幻像
未提交读	是	是	是
提交读	否	是	是
可重复读	否	否	是
可串行读	否	否	否

□ 事务必须运行于可重复读或更高的隔离级别以防止丢失更新

自定义事务隔离级别

- 默认情况下，SQL Server在READ COMMITTED的一个隔离级别上操作
- 若要在应用程序中使用更严格或较宽松的隔离级别，可以通过语句设置会话的隔离级别

语 法：**SET TRANSACTION ISOLATION LEVEL
{ READ COMMITTED | READ UNCOMMITTED
| REPEATABLE READ | SERIALIZABLE }**

- 较低的隔离级别可以增加并发性，但代价是降低数据的正确性
- 相反，较高的隔离级别可以确保数据的正确性，但可能对并发产生负面影响

□动态锁定架构

- 执行查询时，SQL Server基于架构和查询的特性自动确定最合适的锁，这称为动态锁定架构
- SQL Server动态提高和降低锁的粒度和类型
- 在服务器启动时，锁管理器预先分配一定量的锁块和锁拥有者块，并根据sp_configure的设定和SQL Server的版本动态分配锁块和锁拥有者块
- 当发出对锁块的请求时，若没有可用的锁块，则锁管理器自动分配新的锁块，而不是拒绝锁请求

□动态锁定的优点

- SQL Server通过使用适合任务的锁，使系统开销减至最小。同时，数据库管理员也不必再考虑调整锁升级极限



自定义锁定



- SQL Server自动执行锁定，同时也可以通过以下方法自定义应用程序中的锁定：
 - 处理死锁和设置死锁优先级
 - 处理超时和设置锁超时持续时间
 - 设置事务隔离级别
 - 对SELECT、INSERT、UPDATE和DELETE语句使用表级锁定提示
 - 配置索引的锁定粒度

□ SQL Server结束死锁对方法

- SQL Server回滚作为死锁牺牲品的事务
- 通知线程的应用程序（通过返回1205号错误信息）
- 取消线程的当前请求
- 允许其他事务继续进行

□ 将死锁减至最少

- 在所有事务中用同一顺序使用资源
- 尽量减少事务中的步骤，以缩短事务
- 避免使用读取大量行的查询，以缩短事务时间

□自定义锁超时

- 若事务在等待资源时被锁定且导致了死锁，则SQL Server将终止其中参与的一个事务（不涉及超时）
- 如果没有出现死锁，则在其他事务释放锁之前，请求锁的事务被阻塞
- LOCK_TIMEOUT设定允许应用程序设置语句对被阻塞资源的最大等待时间。超过最大等待时间，语句自动取消
- KILL命令可基于服务器进程ID (spid) 中止用户进程



锁定超时



- 使用SET LOCK_TIMEOUT选项，可以设定SQL Server允许事务等待被阻塞的资源释放的最大时间
 - 语法： **SET LOCK_TIMEOUT 超时期限**
- 超时期限是SQL Server返回锁定错误前经过的毫秒数
 - 值为-1 (默认值) 时表示没有超时期限，即无限期等待
 - 值为0时表示根本不等待，并且一遇到锁就返回信息
- 当锁等待超过超时值时，将返回错误
- 使用@@lock_timeout全局变量测定当前会话的锁定超时期限值

9.9 小结

□ 并发操作带来的数据不一致性

- 1. 丢失更新 (Lost Update)
- 2. 不可重复读 (Non-Repeatable Read)
- 3. 读“脏”数据 (Dirty Read)

□ 并发控制机制的任务

- 保证事务的隔离性
- 保证数据库的一致性

□ 可串行化调度的定义

- 可串行化是并发事务正确调度的准则
- 可串行化调度的充分条件

9.9 小结

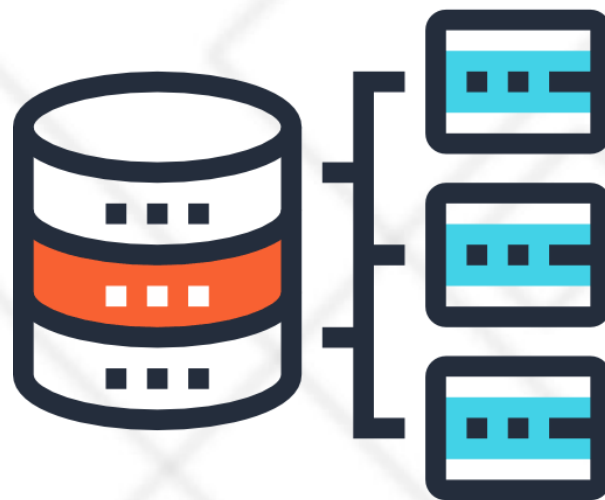
- 数据库的并发控制以事务为单位、使用封锁机制
 - 锁的类型
 - 封锁协议：一级封锁协议 二级封锁协议 三级封锁协议
- 对数据对象施加封锁，带来问题
 - 活锁：先来先服务
 - 死锁：
 - ◆ 预防方法：一次封锁法，顺序封锁法，事务重试法
 - ◆ 死锁的诊断与解除：
 - 超时法、等待图法
 - 解除死锁的方法是回滚一个或多个相关事务，选择一个处理死锁代价最小的事务，将其撤销

9.9 小结

- 两阶段封锁协议
- 多粒度封锁
 - 意向锁
- 并发控制机制调度并发事务操作是否正确的判别准则是可串行性
 - 并发操作的正确性则通常由两阶段封锁协议来保证
 - 两阶段封锁协议是可串行化调度的充分条件，不是必要条件
- 不同的数据库管理系统提供的封锁类型、封锁协议、达到的系统一致性级别不尽相同。但是其依据的基本原理和技术是共同的



北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY



作业

《数据库系统原理》第9章

习题2, 3, 4, 8

□实验4：数据库备份、恢复和权限管理

□实验目的

- 掌握数据库备份以及恢复技术；掌握数据库权限管理技术

□实验内容

- 详见乐学课程

□提交方式

- 打包提交：实验报告、源文件（.sql文件）
- 压缩文件命名方式：学号-姓名-实验X